

```
In [1]: import numpy as np
import scipy.stats as stats
sample_data = np.array([152, 148, 151, 149, 147, 153, 150, 148, 152, 149, 151, 150, 148, 150, 152, 149, 150, 148, 153, 151, 150, 149, 152, 148, 151, 150, 153])
population_mean = 150
sample_mean = np.mean(sample_data)
sample_std = np.std(sample_data, ddof=1)
n = len(sample_data)
z_statistic = (sample_mean - population_mean) / (sample_std / np.sqrt(n))
p_value = 2 * (1 - stats.norm.cdf(np.abs(z_statistic)))
print(f"Sample Mean: {sample_mean:.2f}")
print(f"Z-Statistic: {z_statistic:.4f}")
print(f"P-Value: {p_value:.4f}")
alpha = 0.05
if p_value < alpha:
    print("Reject the null hypothesis: The average weight is significantly different")
else:
    print("Fail to reject the null hypothesis: There is no significant difference")
```

Sample Mean: 150.20

Z-Statistic: 0.6406

P-Value: 0.5218

Fail to reject the null hypothesis: There is no significant difference in average weight from 150 grams.

Lab Experiment 8: Conduct Z test for the Data Given using Python Objective:

To test whether the average weight of a species of birds differs from 150 grams.

Procedure:

1. **Null Hypothesis (H_0):** The average weight of the birds is 150 grams.
2. **Alternative Hypothesis (H_1):** The average weight of the birds is not 150 grams.
3. **Sample:** Measure the weights of 30 birds randomly selected from the population.
4. **Z-Test:** Conduct a Z-test to compare the sample mean to 150 grams.
5. **Decision Rule:** Use a significance level of $\alpha = 0.05$.

Python Code Example:

python

Copy code

```
import numpy as np

import scipy.stats as stats

# Define the sample data (hypothetical weights in grams)

sample_data = np.array([152, 148, 151, 149, 147, 153, 150, 148, 152, 149, 151, 150, 149, 152, 151,
148, 150, 152, 149, 150, 148, 153, 151, 150, 149, 152, 148, 151, 150, 153])

# Population mean under the null hypothesis

population_mean = 150

# Calculate sample statistics

sample_mean = np.mean(sample_data)

sample_std = np.std(sample_data, ddof=1) # Using sample standard deviation

# Number of observations

n = len(sample_data)

# Calculate the Z-statistic

z_statistic = (sample_mean - population_mean) / (sample_std / np.sqrt(n))

# Calculate the p-value
```

Exp No:9 Experiment to understand Matplotlib library use cases in Data Science through visualization

Description: Use Iris data set to understand the Matplot lib library

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
%matplotlib inline
```

```
data=pd.read_csv('/content/Iris_Dataset.csv')
data
```



	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	variety
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa
...	...	...	...	...	...	...
145	146	6.7	3.0	5.2	2.3	Iris-virginica
146	147	6.3	2.5	5.0	1.9	Iris-virginica
147	148	6.5	3.0	5.2	2.0	Iris-virginica
148	149	6.2	3.4	5.4	2.3	Iris-virginica
149	150	5.9	3.0	5.1	1.8	Iris-virginica

150 rows x 6 columns

```
data.info()
```



```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 6 columns):
#   Column          Non-Null Count  Dtype
---  -
0    Id              150 non-null   int64
1    SepalLengthCm   150 non-null   float64
2    SepalWidthCm    150 non-null   float64
3    PetalLengthCm   150 non-null   float64
4    PetalWidthCm    150 non-null   float64
5    variety         150 non-null   object
dtypes: float64(4), int64(1), object(1)
memory usage: 7.2+ KB
```

```
data.describe()
```



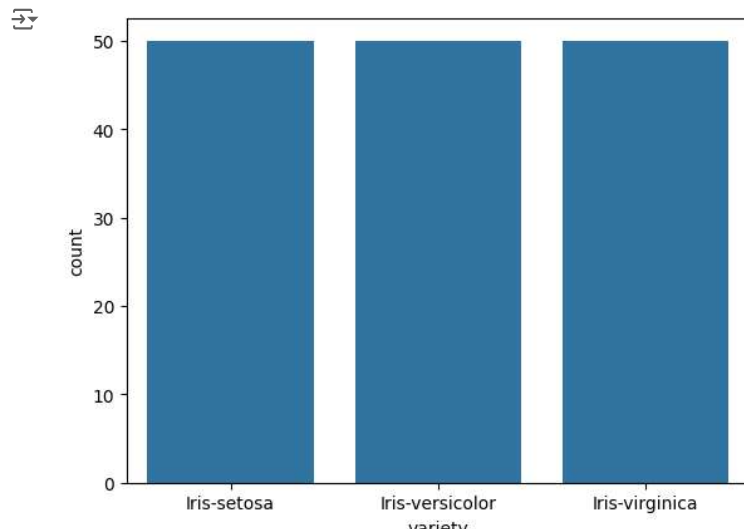
	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm
count	150.000000	150.000000	150.000000	150.000000	150.000000
mean	75.500000	5.843333	3.054000	3.758667	1.198667
std	43.445368	0.828066	0.433594	1.764420	0.763161
min	1.000000	4.300000	2.000000	1.000000	0.100000
25%	38.250000	5.100000	2.800000	1.600000	0.300000
50%	75.500000	5.800000	3.000000	4.350000	1.300000
75%	112.750000	6.400000	3.300000	5.100000	1.800000
max	150.000000	7.900000	4.400000	6.900000	2.500000

```
data.value_counts('variety')
```



count	
variety	
Iris-setosa	50
Iris-versicolor	50
Iris-virginica	50

```
sns.countplot(x='variety',data=data,)
plt.show()
```



```
dummies=pd.get_dummies(data.variety)
```

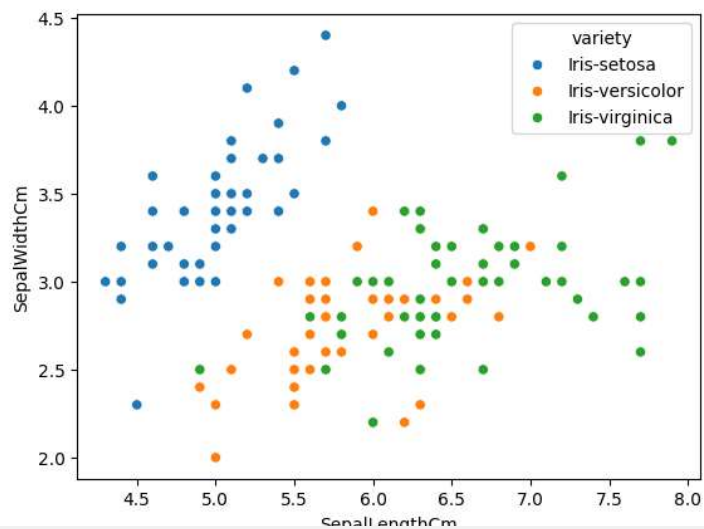
```
FinalDataset=pd.concat([pd.get_dummies(data.variety),data.iloc[:,[0,1,2,3]]],axis=1)
```

```
FinalDataset.head()
```

	Iris-setosa	Iris-versicolor	Iris-virginica	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm
0	True	False	False	1	5.1	3.5	1.4
1	True	False	False	2	4.9	3.0	1.4
2	True	False	False	3	4.7	3.2	1.3
3	True	False	False	4	4.6	3.1	1.5
4	True	False	False	5	5.0	3.6	1.4

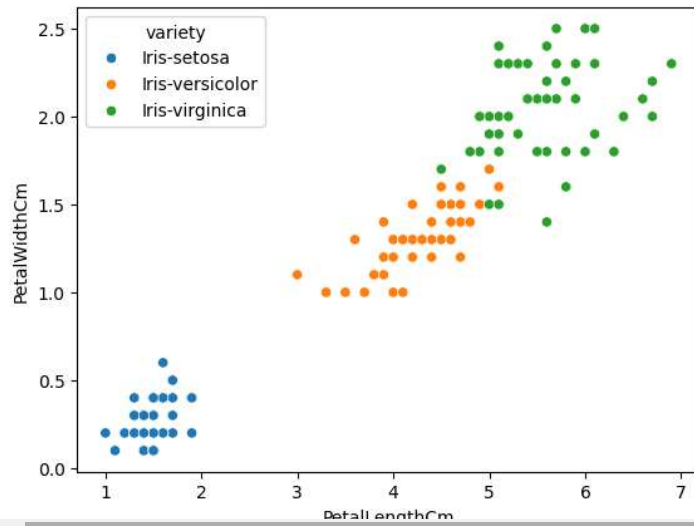
```
sns.scatterplot(x='SepalLengthCm',y='SepalWidthCm',hue='variety',data=data,)
```

```
<Axes: xlabel='SepalLengthCm', ylabel='SepalWidthCm'>
```

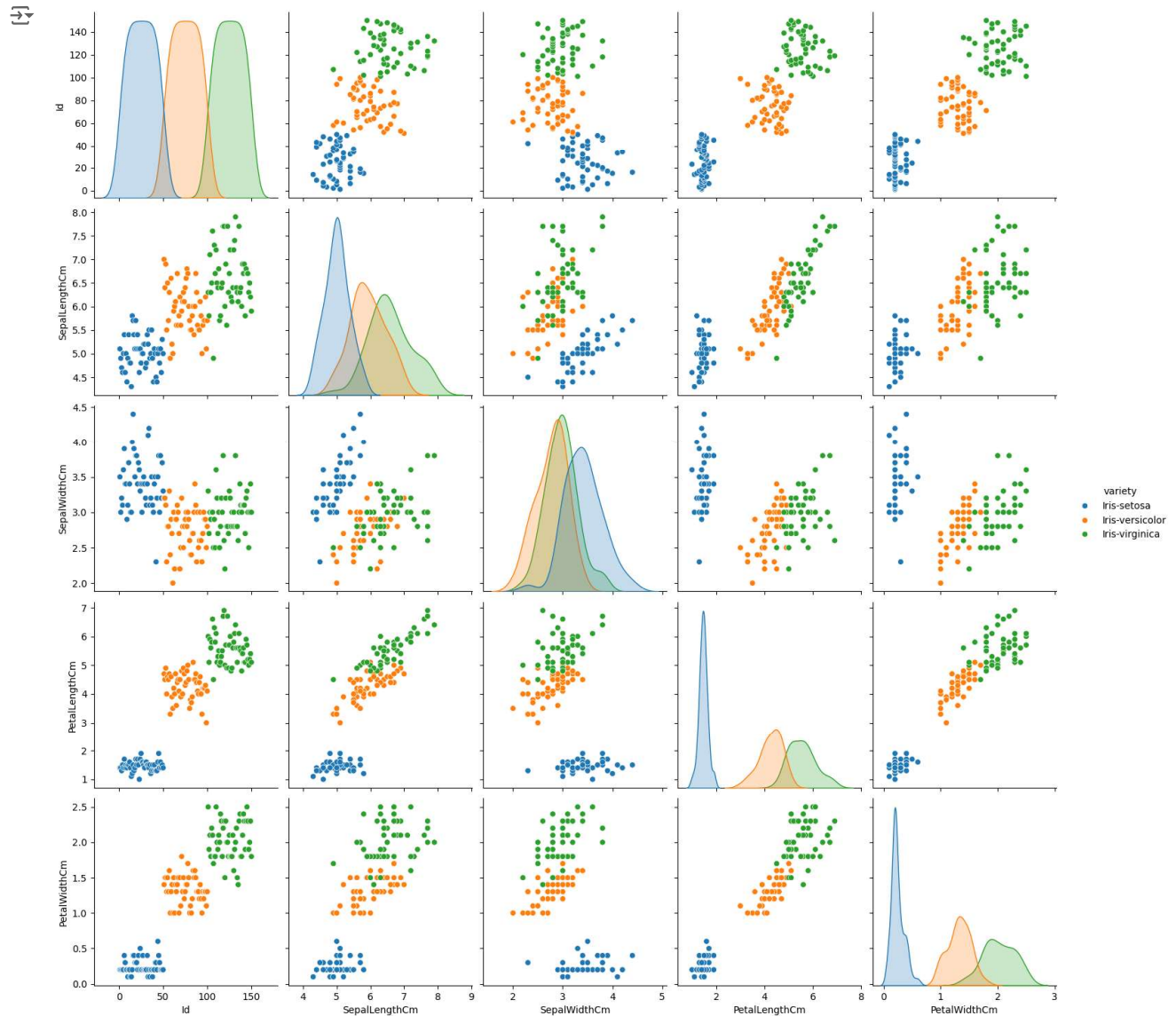


```
sns.scatterplot(x='PetalLengthCm',y='PetalWidthCm',hue='variety',data=data,)
```

```
<Axes: xlabel='PetalLengthCm', ylabel='PetalWidthCm'>
```

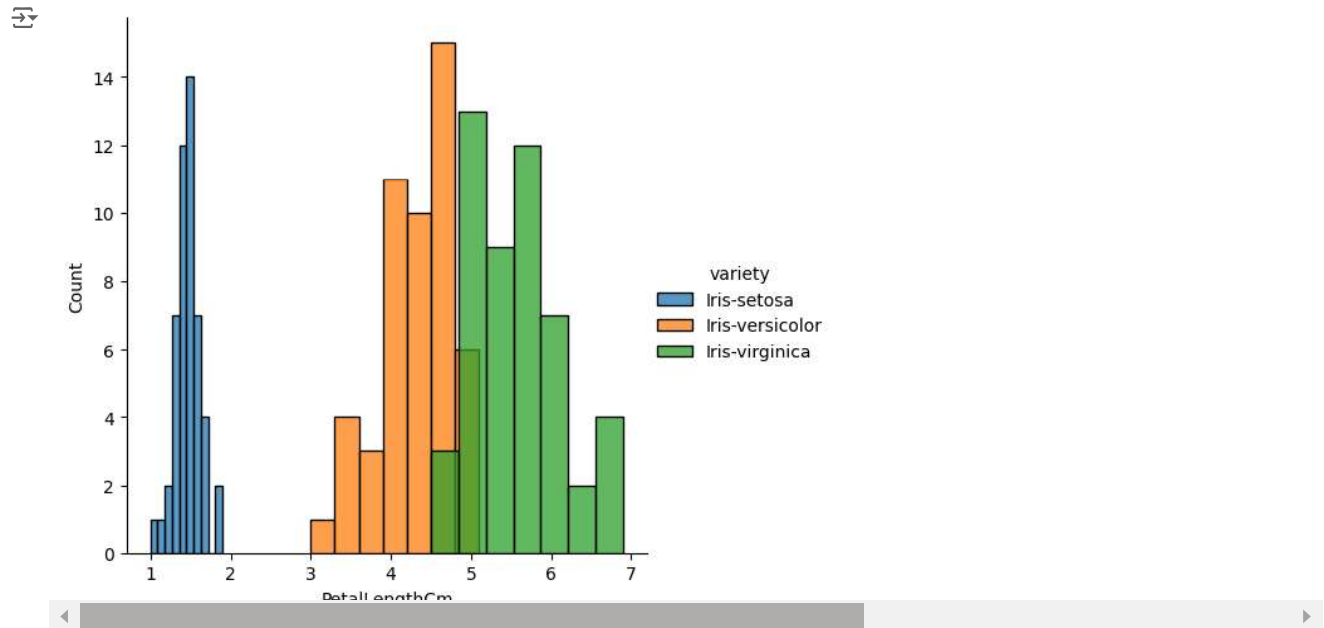


```
sns.pairplot(data,hue='variety',height=3);
```

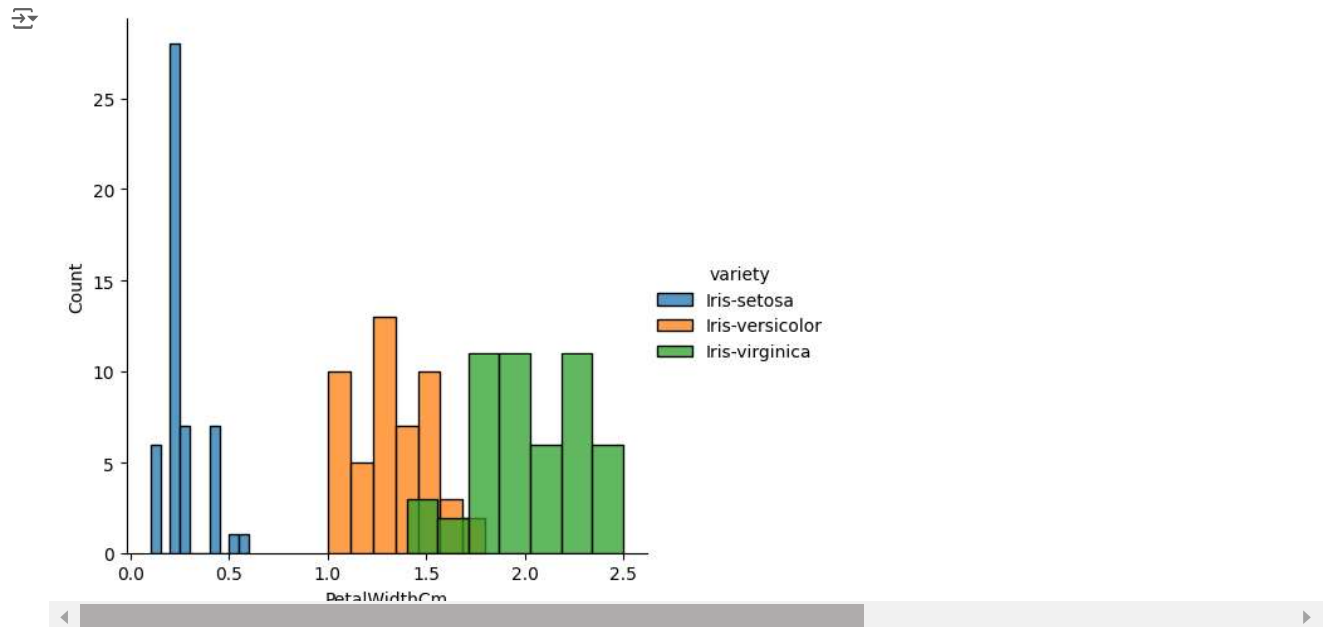


```
plt.show()
```

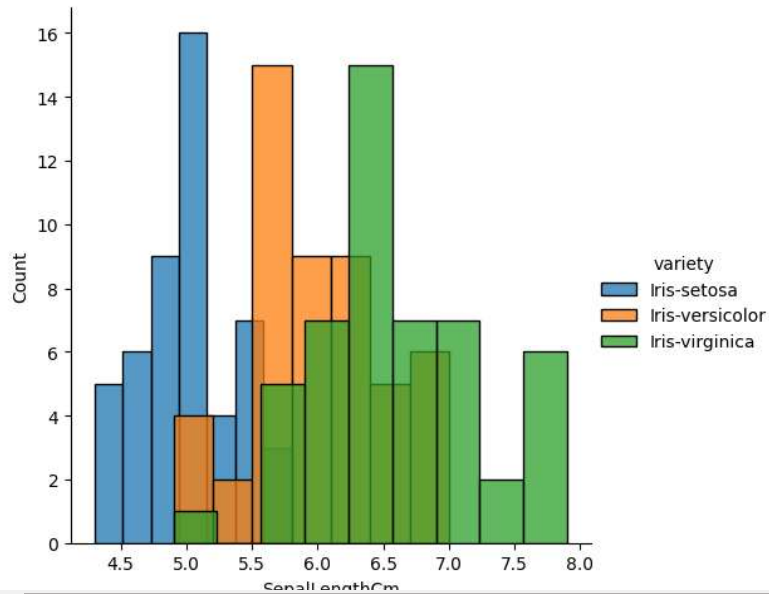
```
sns.FacetGrid(data,hue='variety',height=5).map(sns.histplot,'PetalLengthCm').add_legend();  
plt.show();
```



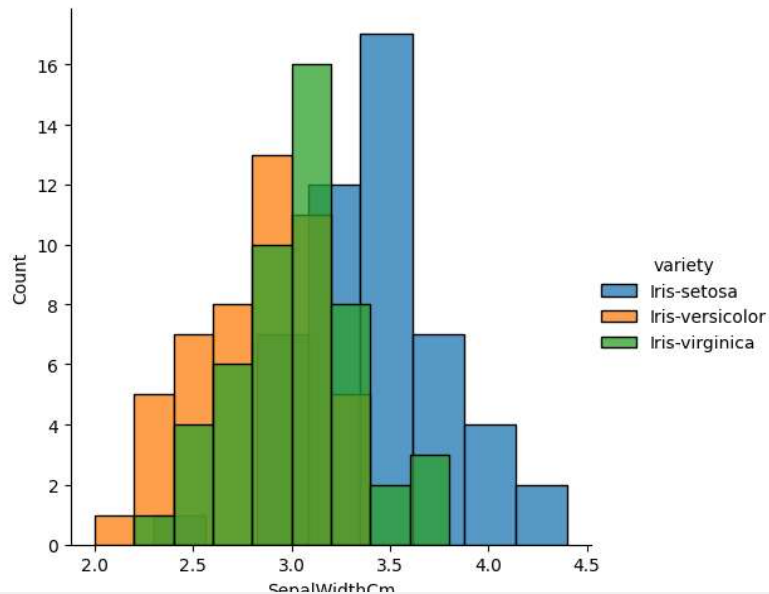
```
sns.FacetGrid(data,hue='variety',height=5).map(sns.histplot,'PetalWidthCm').add_legend();  
plt.show();
```



```
sns.FacetGrid(data,hue='variety',height=5).map(sns.histplot,'SepalLengthCm').add_legend();  
plt.show();
```



```
sns.FacetGrid(data,hue='variety',height=5).map(sns.histplot,'SepalWidthCm').add_legend();  
plt.show();
```



Exp No:10 Experiment to understand array function in Data science.

Description: Understand array function using Numpy library

```
import numpy as np
array=np.random.randint(1,100,9)
array

↵ array([83, 25, 19, 47, 62, 15, 96, 39, 51])

np.sqrt(array)

↵ array([9.11043358, 5.09901951, 4.35889894, 6.8556546 , 7.87400787,
        3.87298335, 9.79795897, 6.244998 , 7.14142843])

array.ndim

↵ 1

new_array=array.reshape(3,3)

new_array

↵ array([[83, 25, 19],
        [47, 62, 15],
        [96, 39, 51]])

new_array.ndim

↵ 2

new_array.ravel()

↵ array([83, 25, 19, 47, 62, 15, 96, 39, 51])

newm=new_array.reshape(3,3)

newm

↵ array([[83, 25, 19],
        [47, 62, 15],
        [96, 39, 51]])

newm[2,1:3]

↵ array([39, 51])

newm[1:2,1:3]

↵ array([[62, 15]])

new_array[0:3,0:0]

↵ array([], shape=(3, 0), dtype=int64)

new_array[0:2,0:1]

↵ array([[83],
        [47]])

new_array[0:3,0:1]

↵ array([[83],
        [47],
        [96]])

new_array[1:3]

↵ array([[47, 62, 15],
        [96, 39, 51]])
```

Exp No:11 Experiment to understand pandas library use cases in Data science.

Description: Understand data frame use cases using pandas library

```
import numpy as np
import pandas as pd
list=[[1,'Smith',50000],[2,'Jones',60000]]
```

```
df=pd.DataFrame(list)
df
```

```
↵
```

	0	1	2
0	1	Smith	50000
1	2	Jones	60000

```
df.columns=['Empd','Name','Salary']
df
```

```
↵
```

	Empd	Name	Salary
0	1	Smith	50000
1	2	Jones	60000

```
df.info()
```

```
↵
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2 entries, 0 to 1
Data columns (total 3 columns):
#   Column  Non-Null Count  Dtype
---  -
0    Empd    2 non-null         int64
1    Name     2 non-null         object
2    Salary   2 non-null         int64
dtypes: int64(2), object(1)
memory usage: 176.0+ bytes
```

```
df=pd.read_csv("/content/50_Startups.csv")
```

```
df.info()
```

```
↵
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 50 entries, 0 to 49
Data columns (total 5 columns):
#   Column                Non-Null Count  Dtype
---  -
0    R&D Spend              50 non-null     float64
1    Administration         50 non-null     float64
2    Marketing Spend        50 non-null     float64
3    State                  50 non-null     object
4    Profit                 50 non-null     float64
dtypes: float64(4), object(1)
memory usage: 2.1+ KB
```

```
df.head()
```

```
↵
```

	R&D Spend	Administration	Marketing Spend	State	Profit
0	165349.20	136897.80	471784.10	New York	192261.83
1	162597.70	151377.59	443898.53	California	191792.06
2	153441.51	101145.55	407934.54	Florida	191050.39
3	144372.41	118671.85	383199.62	New York	182901.99
4	142107.34	91301.77	366168.42	Florida	166187.94

```
df.tail()
```

```
↵
```

	R&D Spend	Administration	Marketing Spend	State	Profit
45	1000.23	124153.04	1903.93	New York	64926.08
46	1315.46	115816.21	297114.46	Florida	49490.75
47	0.00	135426.92	0.00	California	42559.73
48	542.05	51743.15	0.00	New York	35673.41
49	0.00	116983.80	45173.06	California	14681.40

```
import numpy as np
import pandas as pd
df=pd.read_csv("/content/employee.csv")
```

```
df.head()
```



	emp id	name	salary
0	1	SREE VARSSINI K S	5000
1	2	SREEMATHI B	6000
2	3	SREYA G	7000
3	4	SREYASKARI MULLAPUDI	5000
4	5	SRI AKASH U G	8000

```
df.tail()
```



	emp id	name	salary
2	3	SREYA G	7000
3	4	SREYASKARI MULLAPUDI	5000
4	5	SRI AKASH U G	8000
5	6	SRI HARSHAVARDHANAN R	3000
6	7	SRI HARSHAVARDHANAN R	6000

```
df.info()
```



```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7 entries, 0 to 6
Data columns (total 3 columns):
#   Column  Non-Null Count  Dtype
---  ------  -
0   emp id  7 non-null          int64
1   name    7 non-null          object
2   salary  7 non-null          int64
dtypes: int64(2), object(1)
memory usage: 296.0+ bytes
```

```
df.salary
```



	salary
0	5000
1	6000
2	7000
3	5000
4	8000
5	3000
6	6000

```
type(df.salary)
```



```
pandas.core.series.Series
def __init__(data=None, index=None, dtype: Dtype | None=None, name=None, copy: bool | None=None,
fastpath: bool=False) -> None

One-dimensional ndarray with axis labels (including time series).

Labels need not be unique but must be a hashable type. The object
supports both integer- and label-based indexing and provides a host of
methods for performing operations involving the index. Statistical
```

```
df.salary.mean()
```



```
5714.285714285715
```

```
df.salary.median()
```

```
6000.0
```

```
df.salary.mode()
```

```

  salary
0    5000
1    6000

```

```
df.salary.var()
```

```
2571428.5714285714
```

```
df.salary.std()
```

```
1603.5674514745463
```

```
df.describe()
```

```

  emp id    salary
count  7.000000   7.000000
mean    4.000000  5714.285714
std     2.160247  1603.567451
min     1.000000  3000.000000
25%     2.500000  5000.000000
50%     4.000000  6000.000000
75%     5.500000  6500.000000
max     7.000000  8000.000000

```

```
df.describe(include='all')
```

```

  emp id    name    salary
count  7.000000      7   7.000000
unique    NaN      6     NaN
top      NaN  SRI HARSHAVARDHANAN R     NaN
freq     NaN      2     NaN
mean    4.000000      NaN  5714.285714
std     2.160247      NaN  1603.567451
min     1.000000      NaN  3000.000000
25%     2.500000      NaN  5000.000000
50%     4.000000      NaN  6000.000000
75%     5.500000      NaN  6500.000000
max     7.000000      NaN  8000.000000

```

```
empCol=df.columns
```

```
empCol
```

```
Index(['emp id', 'name ', 'salary'], dtype='object')
```

```
emparray=df.values
```

```
emparray
```

```

array([[1, 'SREE VARSSINI K S', 5000],
       [2, 'SREEMATHI B', 6000],
       [3, 'SREYA G', 7000],
       [4, 'SREYASKARI MULLAPUDI', 5000],
       [5, 'SRI AKASH U G', 8000],

```

```
[6, 'SRI HARSHAVARDHANAN R', 3000],  
[7, 'SRI HARSHAVARDHANAN R', 6000]], dtype=object)
```

```
employee_DF=pd.DataFrame(emparray,columns=empCol)
```

```
employee_DF
```



	emp id	name	salary
0	1	SREE VARSSINI K S	5000
1	2	SREEMATHI B	6000
2	3	SREYA G	7000
3	4	SREYASKARI MULLAPUDI	5000
4	5	SRI AKASH U G	8000
5	6	SRI HARSHAVARDHANAN R	3000
6	7	SRI HARSHAVARDHANAN R	6000

Start coding or [generate](#) with AI.

Exp No:12 Experiment to detect outliers in a given data set.

Description: Understand the procedure to identify the outliers in a given dataset


```
#sample calculation for low range(lr) , upper range (ur),percentile
import numpy as np
array=np.random.randint(1,100,16) # randomly generate 16 numbers between 1 to 100
array
```

```
array([27, 50, 44, 6, 58, 61, 23, 86, 67, 20, 75, 7, 79, 61, 90, 54])
```

```
array.mean()
```

```
50.5
```

```
np.percentile(array,25)
```

```
26.0
```

```
np.percentile(array,50)
```

```
56.0
```

```
np.percentile(array,75)
```

```
69.0
```

```
np.percentile(array,100)
```

```
90.0
```

```
#outliers detection
def outDetection(array):
    sorted(array)
    Q1,Q3=np.percentile(array,[25,75])
    IQR=Q3-Q1
    lr=Q1-(1.5*IQR)
    ur=Q3+(1.5*IQR)
    return lr,ur
```

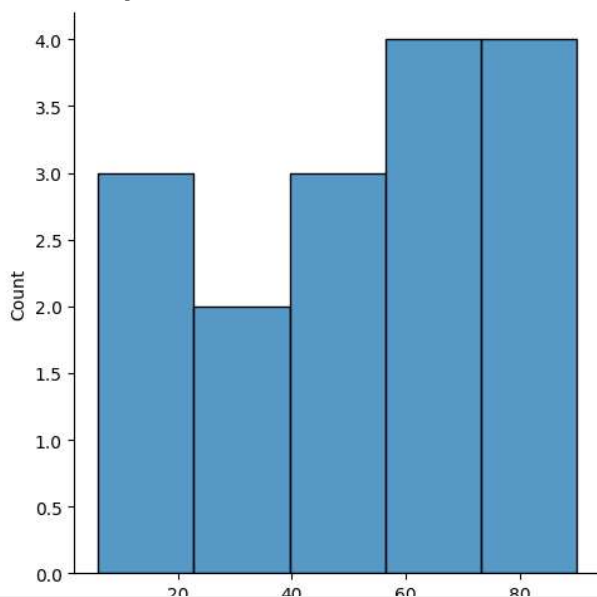
```
lr,ur=outDetection(array)
```

```
lr,ur
```

```
(-38.5, 133.5)
```

```
import seaborn as sns
%matplotlib inline
sns.displot(array)
```

```
<seaborn.axisgrid.FacetGrid at 0x78f3291c2710>
```



```
sns.distplot(array)
```

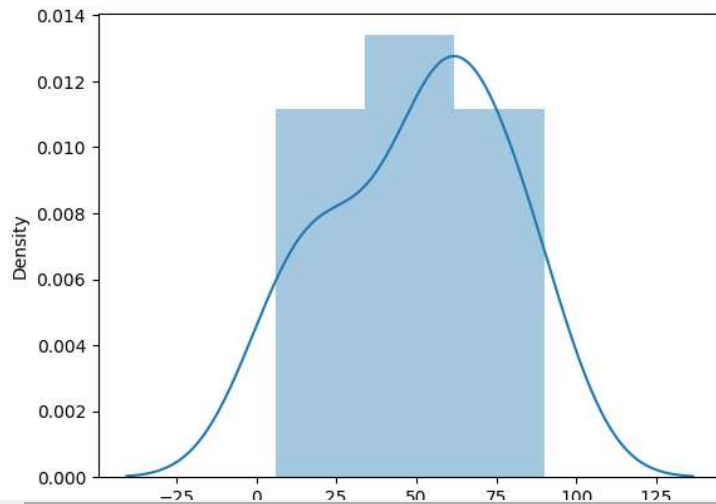
 <ipython-input-19-d72101983c40>:1: UserWarning:

'distplot' is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either 'displot' (a figure-level function with similar flexibility) or 'histplot' (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see <https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751>

sns.distplot(array)
<Axes: ylabel='Density'>

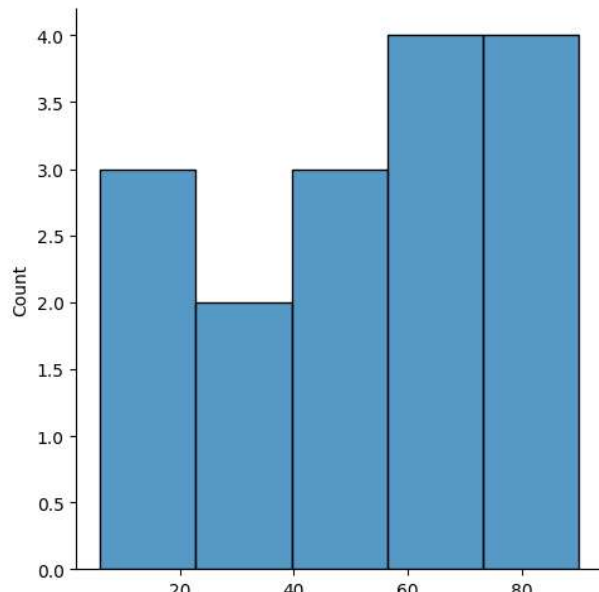


```
new_array=array[(array>lr) & (array<ur)]
new_array
```

 array([27, 50, 44, 6, 58, 61, 23, 86, 67, 20, 75, 7, 79, 61, 90, 54])

```
sns.displot(new_array)
```

 <seaborn.axisgrid.FacetGrid at 0x78f2e09bb580>



```
lr1,ur1=outDetection(new_array)
lr1,ur1
```

 (-38.5, 133.5)

```
final_array=new_array[(new_array>lr1) & (new_array<ur1)]
final_array
```

 array([27, 50, 44, 6, 58, 61, 23, 86, 67, 20, 75, 7, 79, 61, 90, 54])

```
sns.distplot(final_array)
```



<ipython-input-18-7ba96ada5b76>:1: UserWarning:

`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see <https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751>

```
sns.distplot(final_array)  
<Axes: ylabel='Density'>
```

